Retour sur la

Représentation des nombres

Représentation des entiers

Représentation des réels

Opérations arithmétiques

REPRÉSENTATION DES ENTIERS

Représentation des entiers en base B

En base B (B entier > 1), le mot

$\pm d_m d_{m-1} \ldots\ldots d_0$ avec les d_i dans un ensemble de B symboles associés aux valeurs 0,1,2,.....,B-1

représente l'entier dont la valeur est

$$\pm \sum_0^m val(d_i) B^i$$

Remarque : ajouter le symbole qui vaut 0 en fin d'écriture c'est multiplier par B, la valeur de l'entier écrit

Représentation des entiers en base dix

En base dix, l'écriture

$\pm d_m d_{m-1} \ldots\ldots d_0$ où

$\forall i : d_i \in \{$'0', '1', '2', '3, '4', '5', '6', '7', '8', '9 '$\}$

représente l'entier dont la valeur est

$$\pm \sum_0^m val(d_i) dix^i$$

Remarque : ajouter un '0' à la fin, c'est multiplier par dix la valeur de l'entier écrit

Dans la suite, par défaut toutes les écritures d'entiers sont en base dix

Entiers représentables

- Si on dispose de n bits pour coder un entier naturel, on peut représenter 2^n entiers.
- Par exemple :

  les entiers compris entre 0 et 2^n -1.

  – Pour n=16 : 0 à 65 535
  – Pour n=32 : 0 à 4 294 967 295
  – Pour n=64 : 0 à environ $1.8\ 10^{19}$

Négatifs
Complément à 2 (exemple sur 8 bits)

- Il faut aussi coder des entiers négatifs : donc au lieu de coder les entiers de 0 à $2^8 – 1$, on code de :

- 2^7 à -1 et de 0 à $2^7 – 1$

- Pour cela, on considère que l'on manipule (écrit) des entiers modulo 2^8 et pour toute écriture ***e*** :

Négatifs
Complément à 2 (exemple sur 8 bits)

- pour toute écriture ***e*** , si le bit de poids fort est
  - 0 : ***e*** code l'entier ***x*** positif (ou nul) dont l'écriture en base 2 est ***e*** , donc ***x*** est compris entre 0 à $2^7 - 1$
  - 1 : ***e*** code l'entier (strictement) négatif dont la valeur ***x*** est congrue modulo 2^8 à l'entier positif ***y*** dont l'écriture en base 2 est ***e*** : y n'est pas égal à |x| et x = y - 2^8 (qui est bien négatif et en outre compris entre - 2^7 et -1).
- Quelle est la valeur de la valeur absolue de ***x*** : |***x***| ?

Négatifs
Complément à 2 (exemple sur 8 bits)

- Quelle est la valeur de la valeur absolue de ***x*** : |***x***| ?
- Comme 2^8 = ***y - x***, on a :
  2^8 = ***y + |x|***,
  soit écrit en base 2 :
  (Res1) 100000000 = ***e*** + ecriture(|***x***|)
- Or si on note pour toute écriture E :
  E' le complément à 1 de E, on a :
  E + E' = 11111111
  Et donc : E + E' + 1 = 100000000
  Donc |x| s'écrit ***e'*** +1
  (d'où *la recette de cuisine* pour passer de l'écriture d'un entier à l'écriture de son opposé)

Négatifs
Complément à 2 (exemple sur 8 bits)

- Pour les entiers négatifs on prend le complément à 1 bit à bit auquel on ajoute 1 :
  - 24 est codé : `00011000`
  - le complément à 1 : `11100111`
  - on ajoute 1 pour obtenir le codage de -24 :
    `11101000`
  - Et on peut aussi partant de -24 retrouver ainsi sa valeur absolue : 24

Négatifs
Complément à 2 (exemple sur 8 bits)

- Comme on travaille dans les entiers modulo 2^8 tous les calculs se passent « bien » (et c'est l'intérêt de tout ça, sinon pourquoi faire compliqué quand on peut faire simple !)
- Pratique, parce que :
  - ça additionne bien : une seule opération (un seul circuit) pour coder l'addition quel que soit le signe des entiers additionnés :

```
24+(-1)   = 00011000 + 11111111 = (0)00010111
24+(-24)  = 00011000 + 11101000 = (1)00000000
```

  - Un seul codage pour zéro

Types entiers en java, codés en complément à deux

	# bits	Min	Max
short	8 bits	-2^{15} = -32768	$2^{15}-1$ = 32767
int	16 bits	-2^{31} = -2 147 483 648	$2^{31}-1$ = 2 147 483 647
long	32 bits	-2^{63} = -9 223 372 036 854 775 808	$2^{63}-1$ = 9 223 372 036 854 775 807

Addition de deux entiers naturels (codés en complément à 2 sur 8 bits)

```
    0 0001010          10
+   0 0001111      +   15
=   0 0011001      =   25

    0 1111010          positif
+   0 1101111      +   positif
=   1 1101001      =   négatif
```

Overflow

Le résultat n'est égal que sur un bit de plus !

Addition de deux entiers négatifs (codés en complément à 2 sur 8 bits)

Même problème, le résultat est faux si la valeur absolue de la somme est trop grande

Dans les deux cas le résultat serait correct sur n+1 bits

Dans les deux cas, le résultat est faux si et seulement si le bit de signe est modifié

Addition de deux entiers de signes contraires

```
        11111111          -1
+       00010100    +     20
= (1)   00010011    =     19
```

Résultat toujours correct, (mais faux sur n+1 bits)

REPRÉSENTATION DES RÉELS VIRGULE FIXE

La virgule virtuelle

c.à.d. division par 2^n

Représentation des réels en base B

En base B (B entier > 1), le mot

$\pm d_m d_{m-1} \ldots\ldots d_{-n}$ avec les d_i dans un ensemble de B symboles associés aux valeurs 0,1,2,.....,B-1

représente le réel dont la valeur est

$$\pm \sum_{-n}^{m} val(d_i) B^i$$

~~Remarque : ajouter le symbole qui vaut 0 en fin d'écriture c'est multiplier par B, la valeur de l'entier écrit~~

Représentation des réels en base dix

En base dix, l'écriture

$\pm d_m d_{m-1} \ldots\ldots d_{-n}$ où

$\forall i : d_i \in \{$'0', '1', '2', '3, '4', '5', '6', '7', '8', '9 '$\}$

représente l'entier dont la valeur est

$$\pm \sum_{-n}^{m} val(d_i) dix^i$$

~~Remarque : ajouter un '0' à la fin, c'est multiplier par dix la valeur de l'entier écrit~~

Dans la suite, par défaut toutes les écritures d'entiers sont en base dix

Réels représentables

- Si on dispose de n bits pour coder un réel, on peut représenter 2^n réels.
- Par exemple :
  - Pour n=16 : 0 à 65 535
  - Pour n=32 : 0 à 4 294 967 295
  - Pour n=64 : 0 à environ $1.8\ 10^{19}$

Exemple avec 3 bits

écriture	n=0	n=1	n=2
000	0	0	0
001	1	0,5	0,25
010	2	1	0,5
011	3	1,5	0,75
100	-4	-2	-1
101	-3	-1,5	-0,75
110	-2	-1	-0,5
111	-1	-0,5	-0,25

Exemple avec 5 bits (m=n=2)

bin	dec	bin	dec	bin	dec	bin	dec
00000	0,00	01000	2,00	10000	-4,00	11000	-2,00
00001	0,25	01001	2,25	10001	-3,75	11001	-1,75
00010	0,50	01010	2,50	10010	-3,50	11010	-1,50
00011	0,75	01011	2,75	10011	-3,25	11011	-1,25
00100	1,00	01100	3,00	10100	-3,00	11100	-1,00
00101	1,25	01101	3,25	10101	-2,75	11101	-0,75
00110	1,50	01110	3,50	10110	-2,50	11110	-0,50
00111	1,75	01111	3,75	10111	-2,25	11111	-0,25

Négatifs
Complément à 2 (exemple sur 8 bits)

- Pour les réels négatifs on prend le complément à 1 bit à bit auquel on ajoute 1 (ici avec m=3 et n=4, donc 2^{-4}) :
  - 2,375 est codé : `00100110`
  - le complément à 1 : `11011001`
  - on ajoute 1 (2^{-4}) pour obtenir -2,375 : `11011010`

- Ça additionne toujours bien :

```
1,5 +(-2,375) = 00011000 + 11011010 = (1)11110010
1,5 +(-1,5)   = 00011000 + 11101000 = (1)00000000
```